

Meridian

System Design Document

Multi-topic AI-powered blogging platform

Microservices Architecture · Vue 3 Frontend · AI Agent · TTS

Stack	NestJS (×4 services) · Vue 3 · SQLite · Tailwind CSS v4
AI Features	GPT-4o research · DALL-E / FLUX / Gemini / Unsplash images · Kokoro TTS
Deployment	HuggingFace Spaces · Docker · Supervisor · GitHub Actions CI/CD
Agent	Publishes 2 blog posts/week via automated cron (Sat + Sun 3am)
A/B Testing	Two brand layouts — Editorial Light (A) & Dark Magazine (B)
Date	June 2026

1. System Overview

Meridian is a multi-topic blogging platform built on a microservices backend and a Vue 3 single-page application frontend. It features an autonomous AI agent that publishes two research-backed articles per week, a text-to-speech reader powered by a local Kokoro AI model, an A/B layout testing system that randomly assigns one of two brand identities per browser session, and a full-featured admin panel for manual content management.

All services run inside a single Docker container orchestrated by Supervisord, hosted on HuggingFace Spaces and deployed automatically via GitHub Actions on every push to main.

2. System Architecture

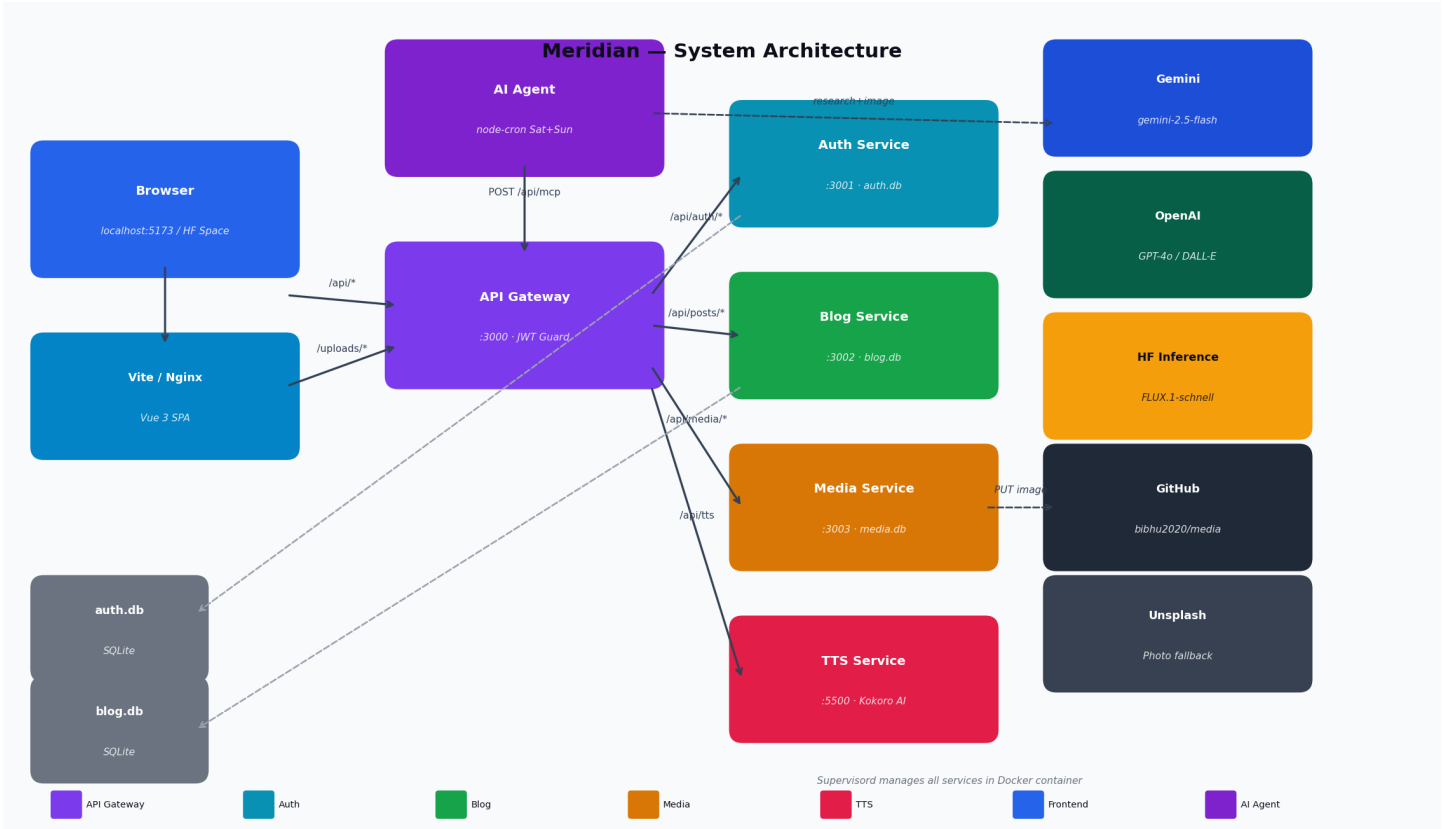


Figure 0: Full system architecture — browser through microservices to external AI APIs

3. Component Descriptions

Component	Port	DB	Responsibility
API Gateway	3000	—	Single entry point for all browser requests. Validates JWTs (JwtStrategy + PassportMod...
Auth Service	3001	auth.db	User accounts, bcrypt password hashing, JWT signing and validation. Issues 24h access...
Blog Service	3002	blog.db	Posts, categories, tags, comments, view counters. Seeds sample data on empty DB. Tag...
Media Service	3003	media.db	Multer file upload handler. Uploads to GitHub CDN repo; falls back to local /uploads/ pat...
TTS Service	5500	—	Kokoro AI local inference model. Accepts POST /tts { text } and returns WAV audio blob.

AI Agent	—	—	node-cron scheduled jobs. Researches topics via GPT-4o web search, writes articles, ge
Vue 3 Frontend	5173	—	SPA with Vue Router, Pinia stores (auth, blog, layout), TipTap editor, TTS player, A/B lay
Nginx	80	—	Reverse proxy inside Docker. Routes /api/* to API Gateway and /uploads/* to Media Ser

4. Data Model

Each NestJS service owns its own SQLite database with TypeORM synchronize:true (auto-migrates on startup).
Key entities:

Table	Service	Key Columns
users	auth.db	id, name, email, password (bcrypt), role, createdAt
posts	blog.db	id, title, slug, content (HTML), excerpt, featuredImage, categoryId, authorI
categories	blog.db	id, name, slug, description, color (#hex), icon (emoji)
tags	blog.db	id, name, slug
posts_tags_tags	blog.db	postsId (FK posts), tagsId (FK tags) – junction table
comments	blog.db	id, postId, authorName, authorEmail, content, status (pending/approved)
media	media.db	id, filename, originalName, mimeType, url, userId, alt, createdAt

5. API Gateway Pattern

The API Gateway is the **sole public entry point**. It validates the JWT before forwarding any authenticated request. Internal services trust the forwarded Authorization: Bearer <token> header and run their own JwtStrategy independently — there is no shared auth library.

Route Pattern	Forwarded To	Auth Required
POST /api/auth/*	Auth Service :3001	No
GET /api/posts/*	Blog Service :3002	No
POST /api/posts/*	Blog Service :3002	Yes (admin)
GET /api/categories, /api/tags	Blog Service :3002	No
POST /api/comments/*	Blog Service :3002	No
POST /api/media/upload	Media Service :3003	Yes (admin)
POST /api/mcp	MCP Controller	Yes (agent JWT)
POST /api/tts	TTS Service :5500	No

6. Application Flow Sequence Diagrams

The following sequence diagrams document every significant user-facing and system-level flow in Meridian. Solid arrows indicate synchronous calls; dashed arrows indicate asynchronous or return flows. Yellow sticky notes show internal decisions or side-effects.

6.1 User Registration

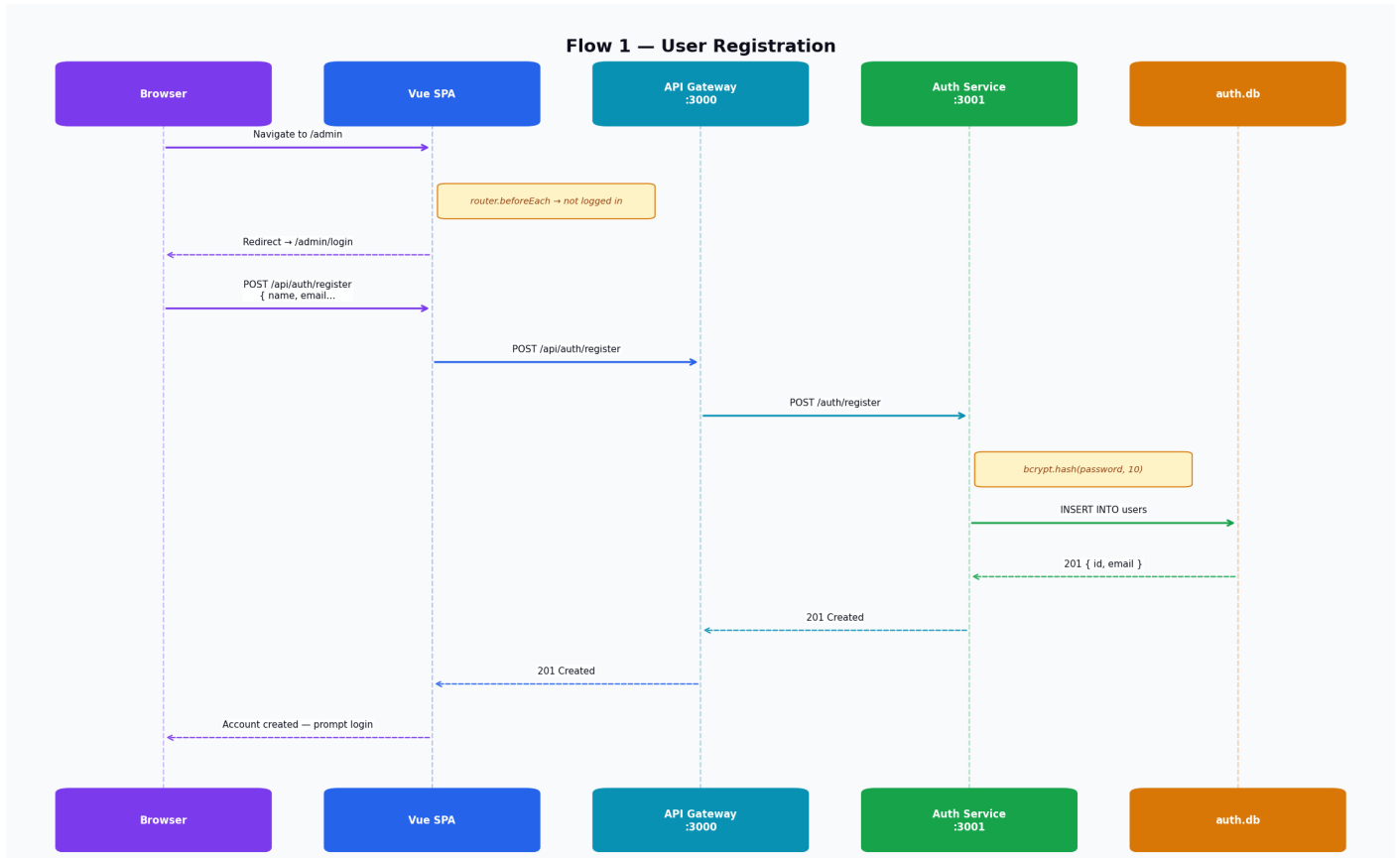


Figure 1: User registration — password is bcrypt-hashed before storage in auth.db

6.2 User Login / JWT Issue

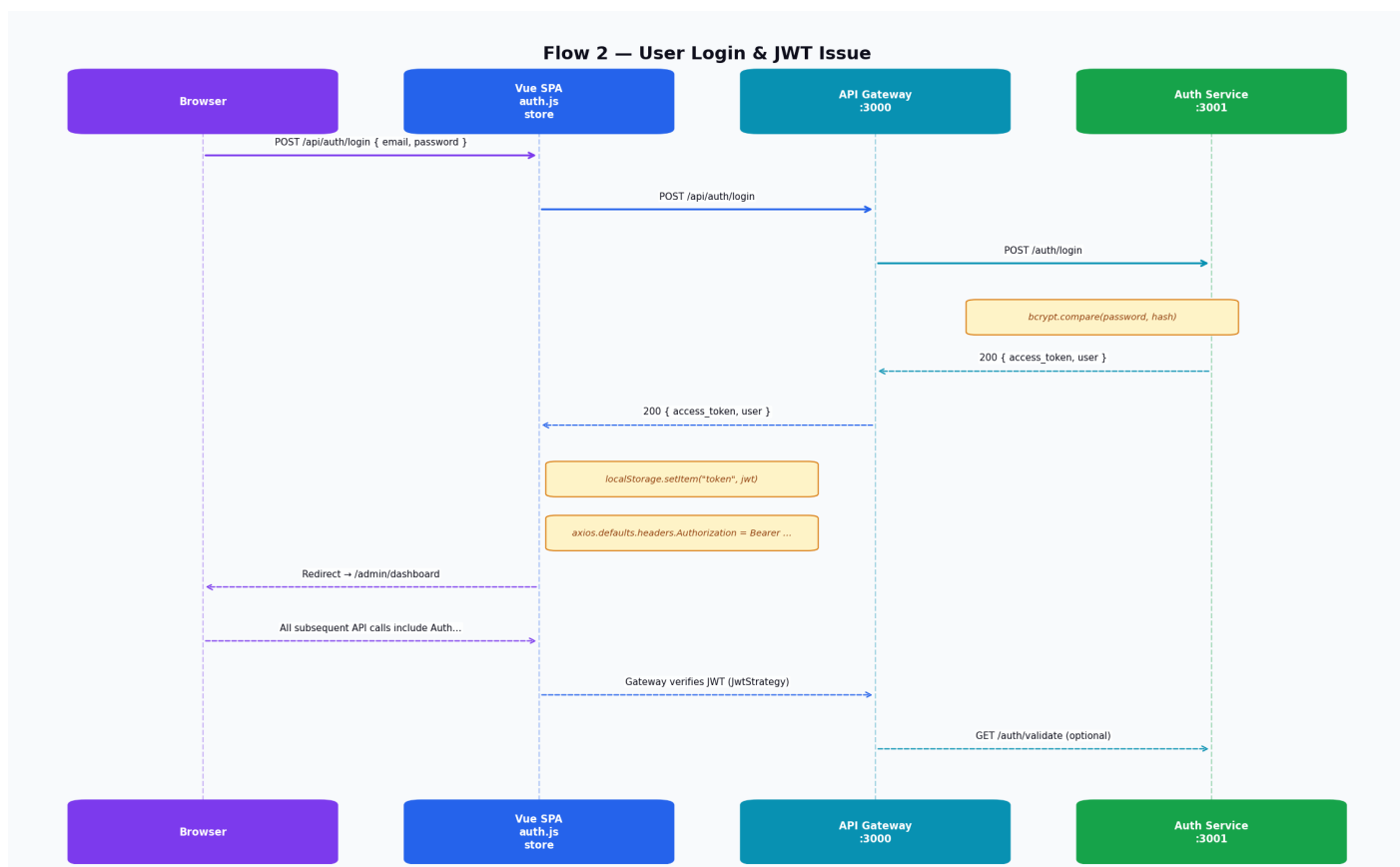


Figure 2: JWT issued by Auth Service; stored in localStorage; auto-attached to all API requests via Axios default header

6.3 A/B Layout Assignment

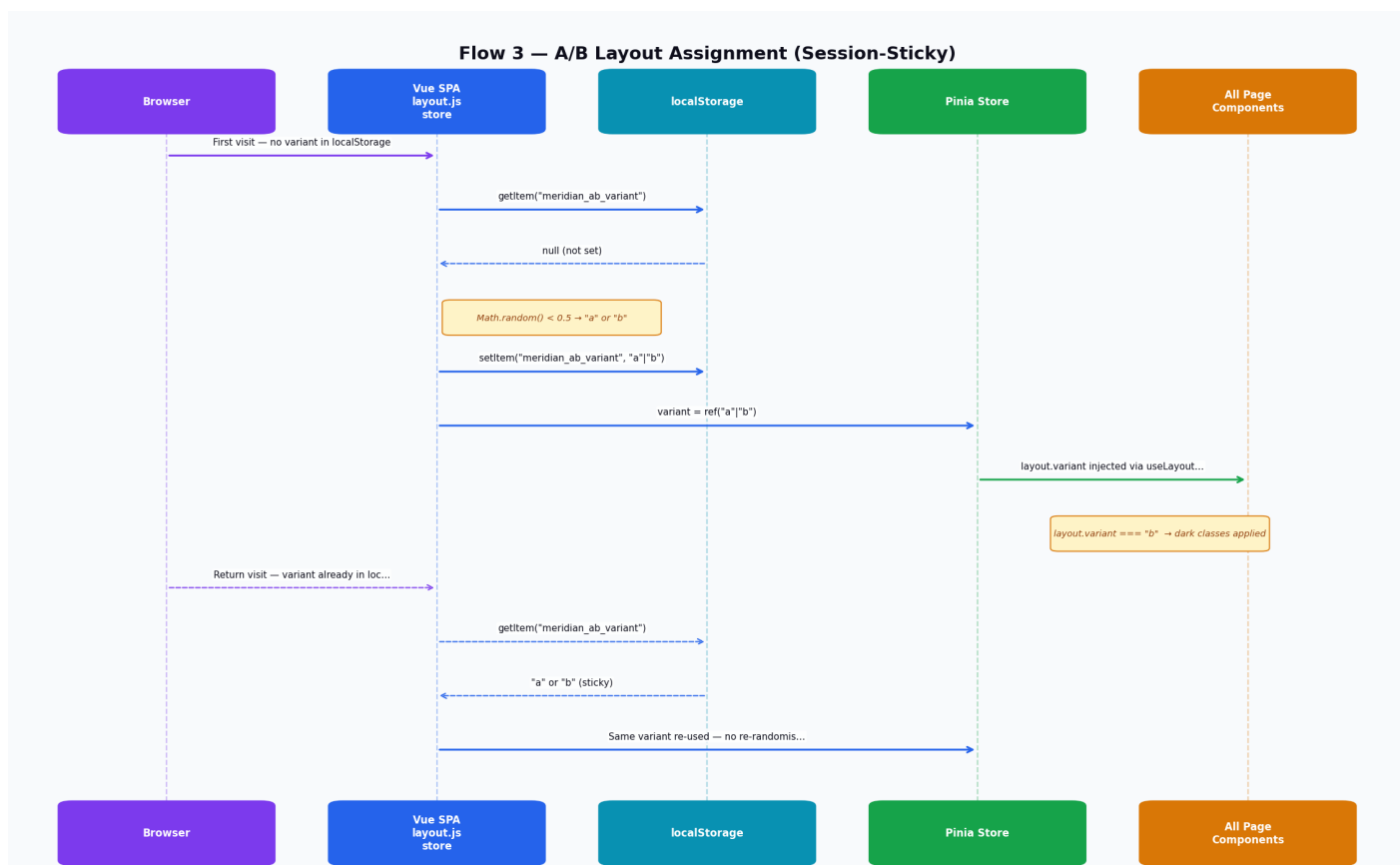


Figure 3: A/B variant randomly assigned once per browser; persisted in localStorage so the same user always sees the same layout

6.4 Browse Home / All Posts

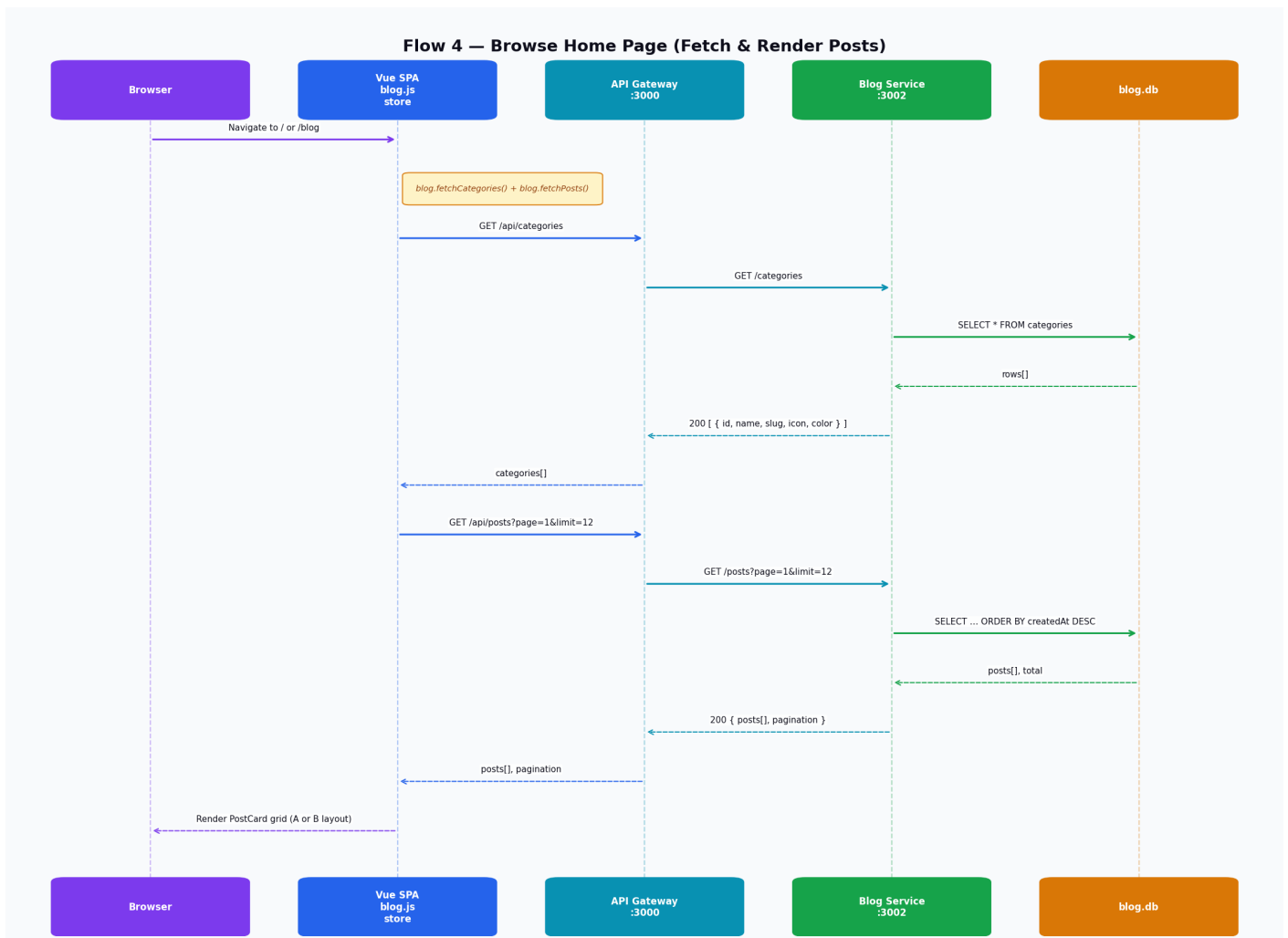


Figure 4: Posts are fetched lazily by the Pinia blog store; results are cached and re-used across components

6.5 Read Blog Post

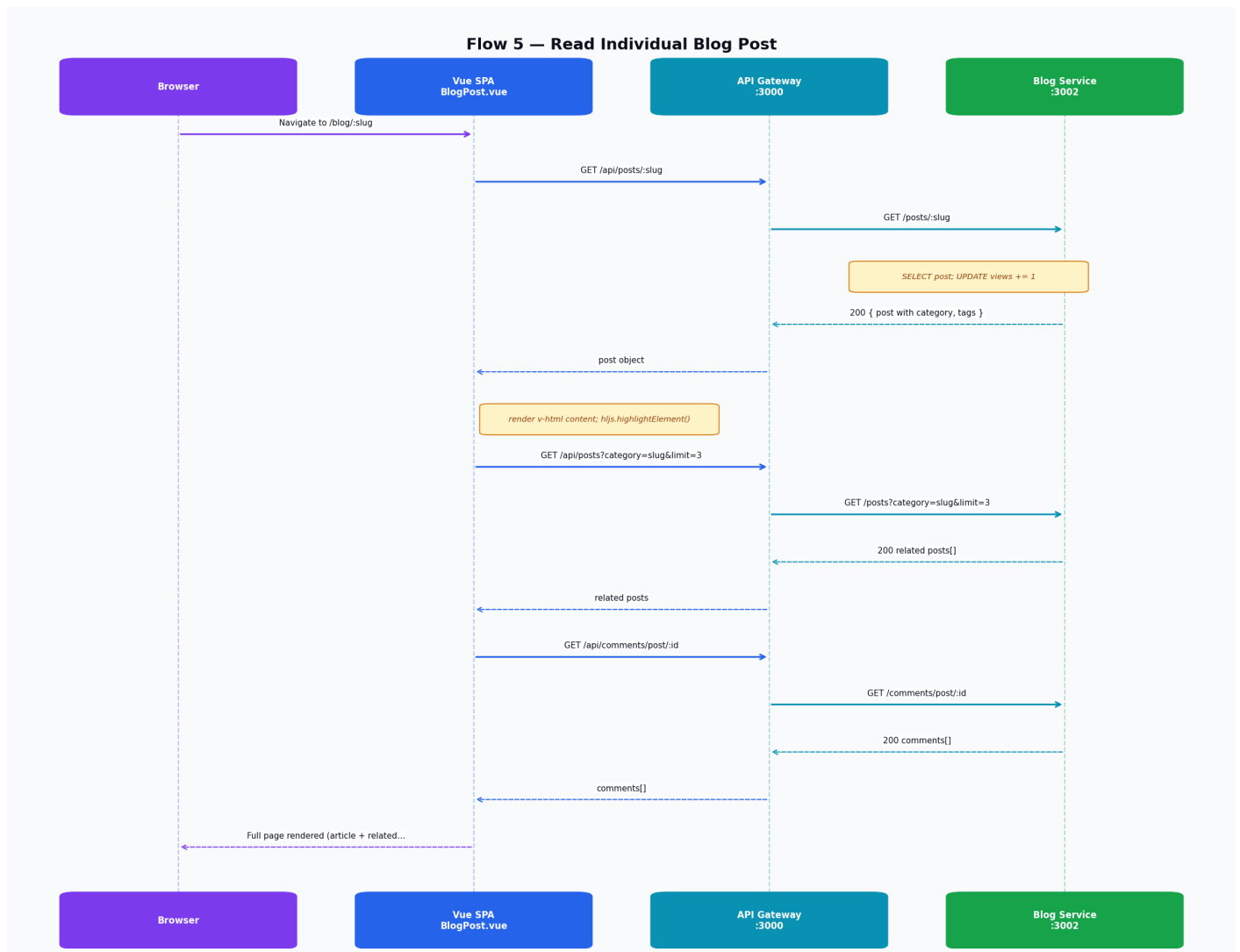


Figure 5: Each post view increments the view counter in *blog.db*; related posts and comments are loaded in parallel

6.6 Text-to-Speech Playback

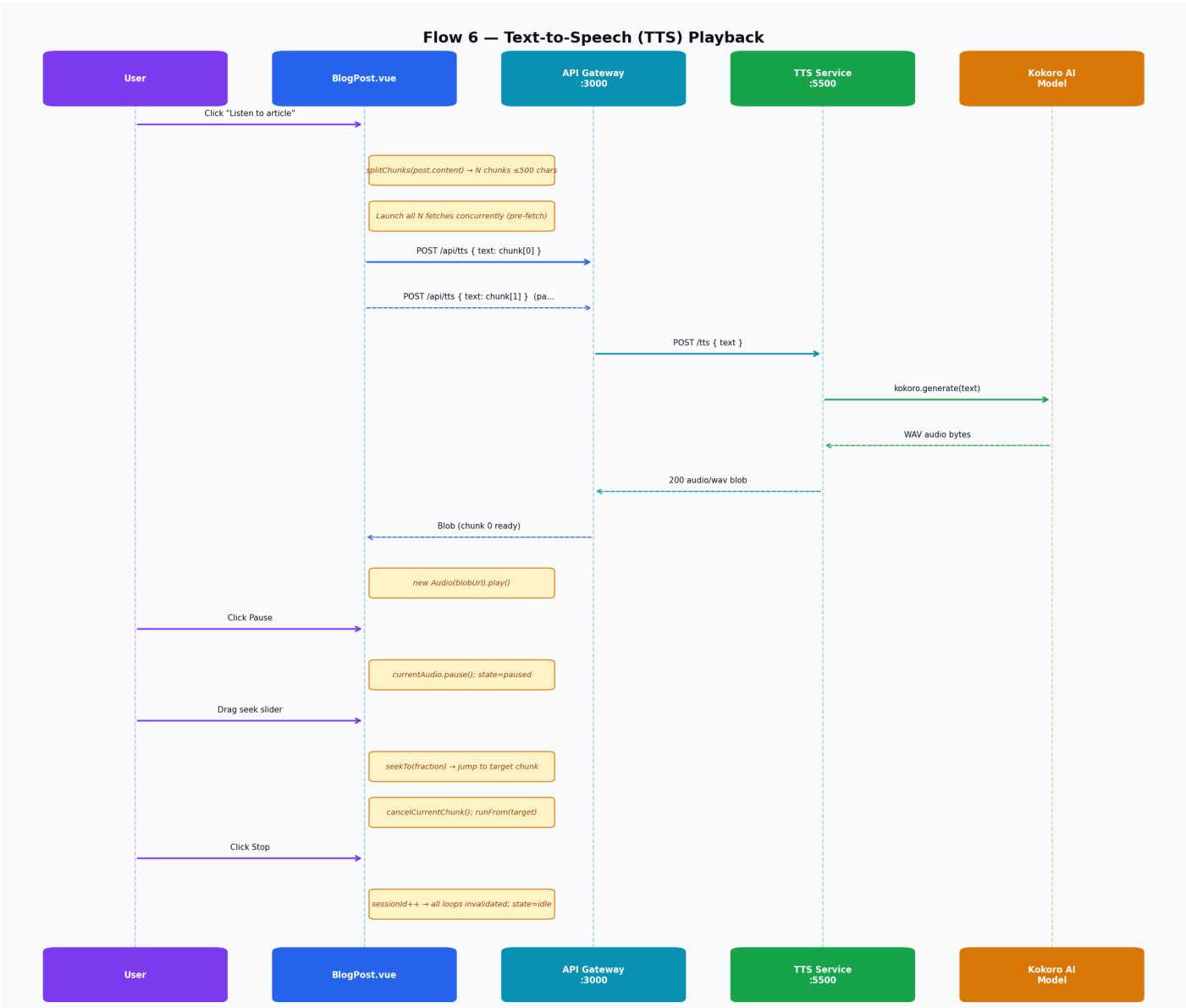


Figure 6: All TTS chunks are pre-fetched concurrently; seek works with already-fetched blobs. Kokoro AI runs locally in the Docker container.

6.7 Search Articles

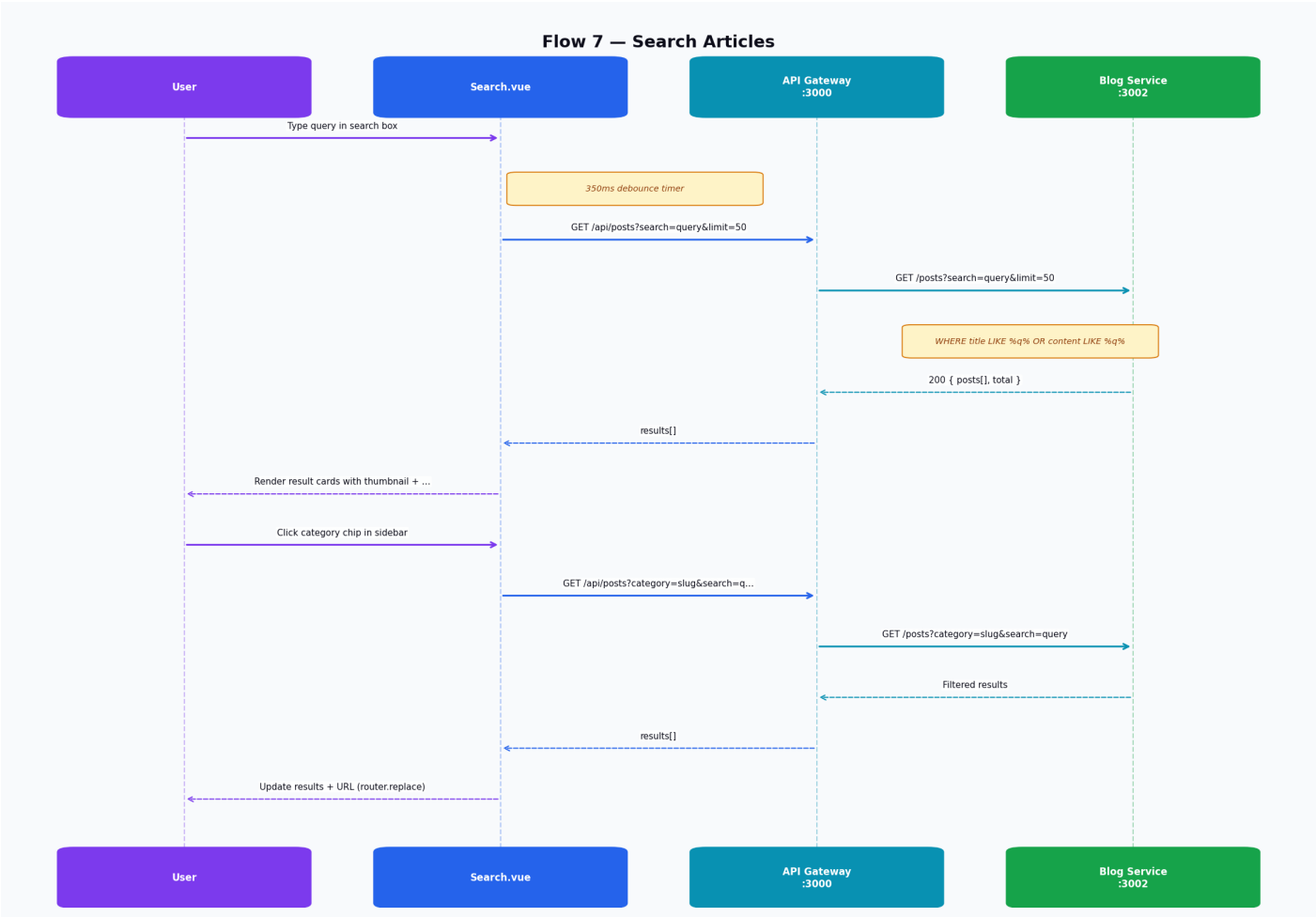


Figure 7: Search is fully client-side driven with a 350ms debounce; URL query params updated so search links are shareable

6.8 Submit Comment

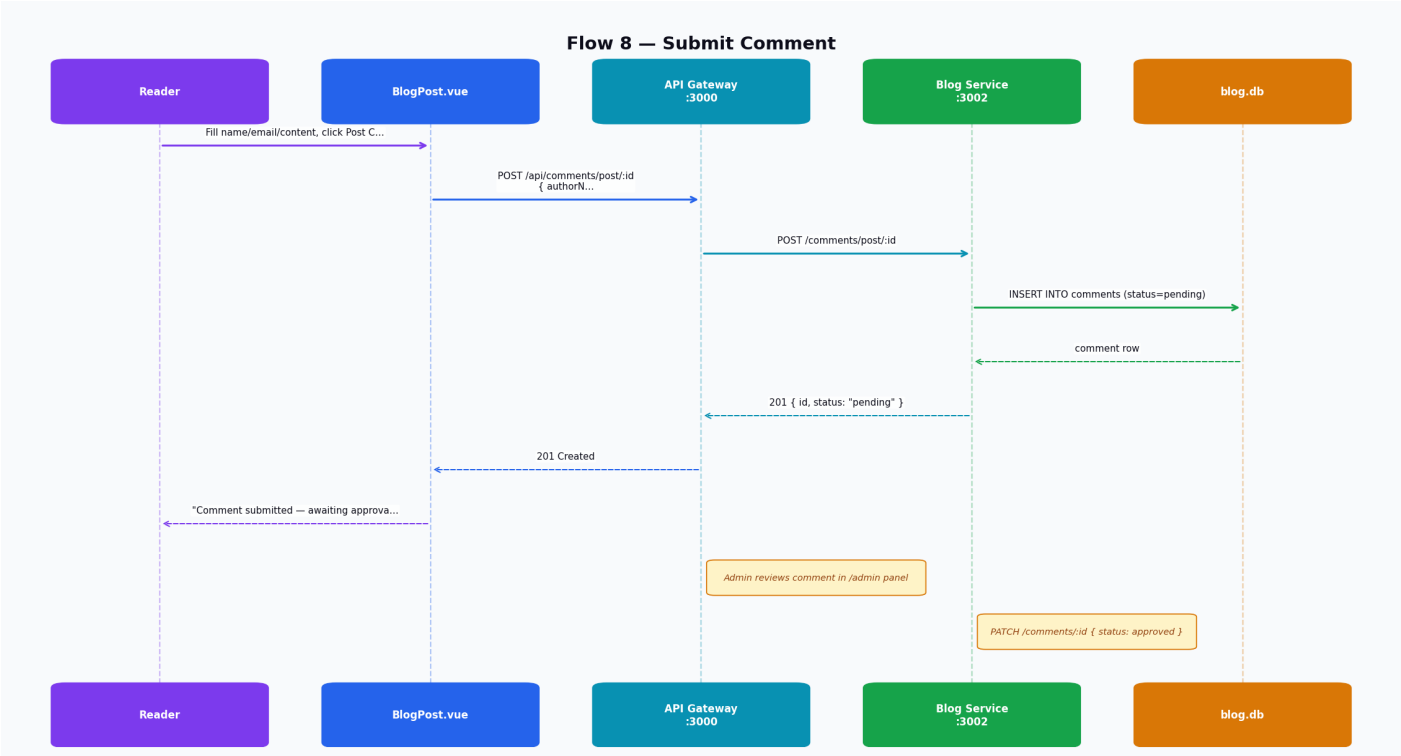


Figure 8: Comments are stored with status=pending; admin approves via the admin panel before they appear publicly

6.9 Admin Create Blog Post

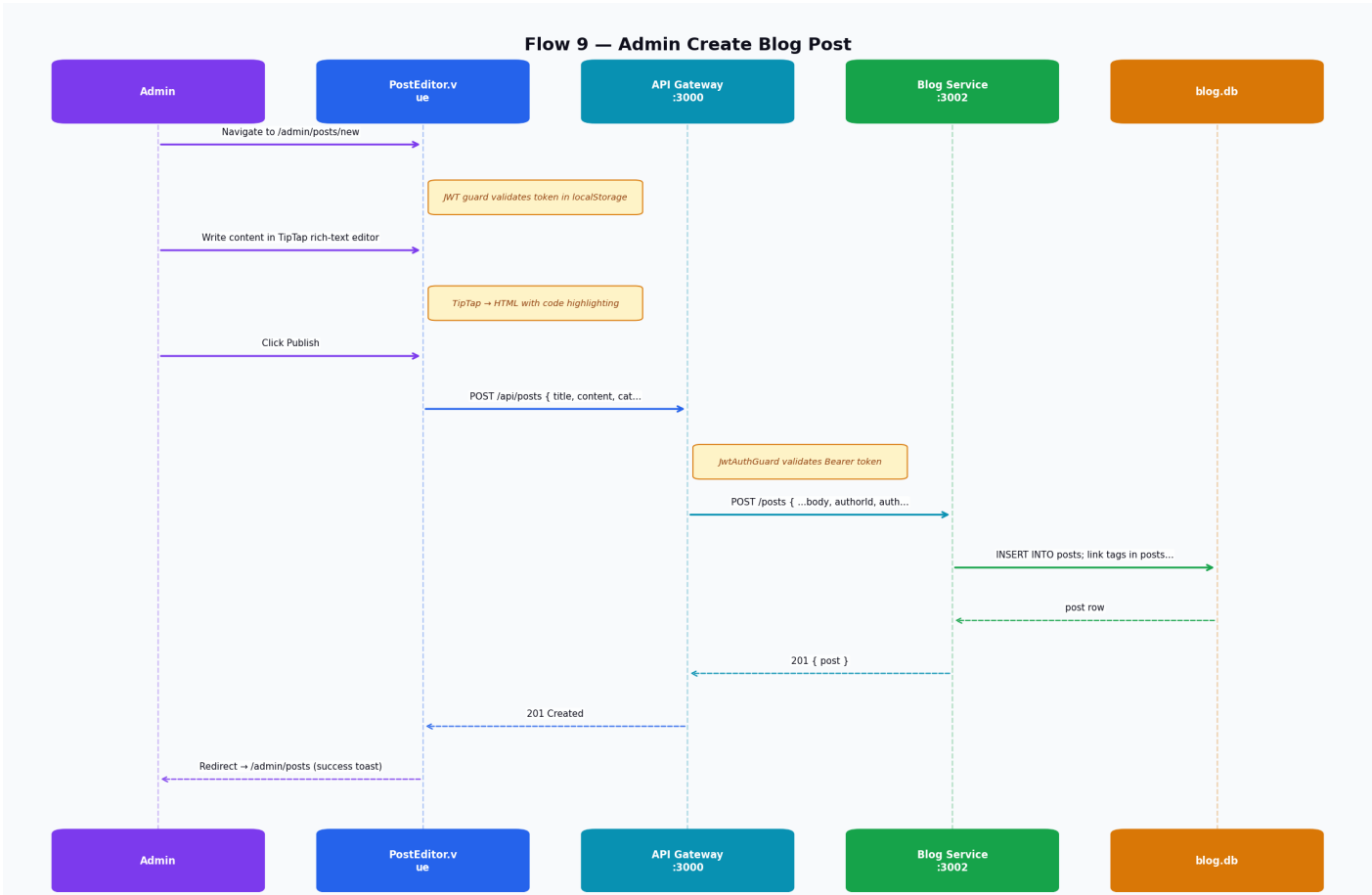


Figure 9: Admin JWT is validated at the API Gateway before forwarding; tags use a junction table posts_tags_tags

6.10 Media File Upload

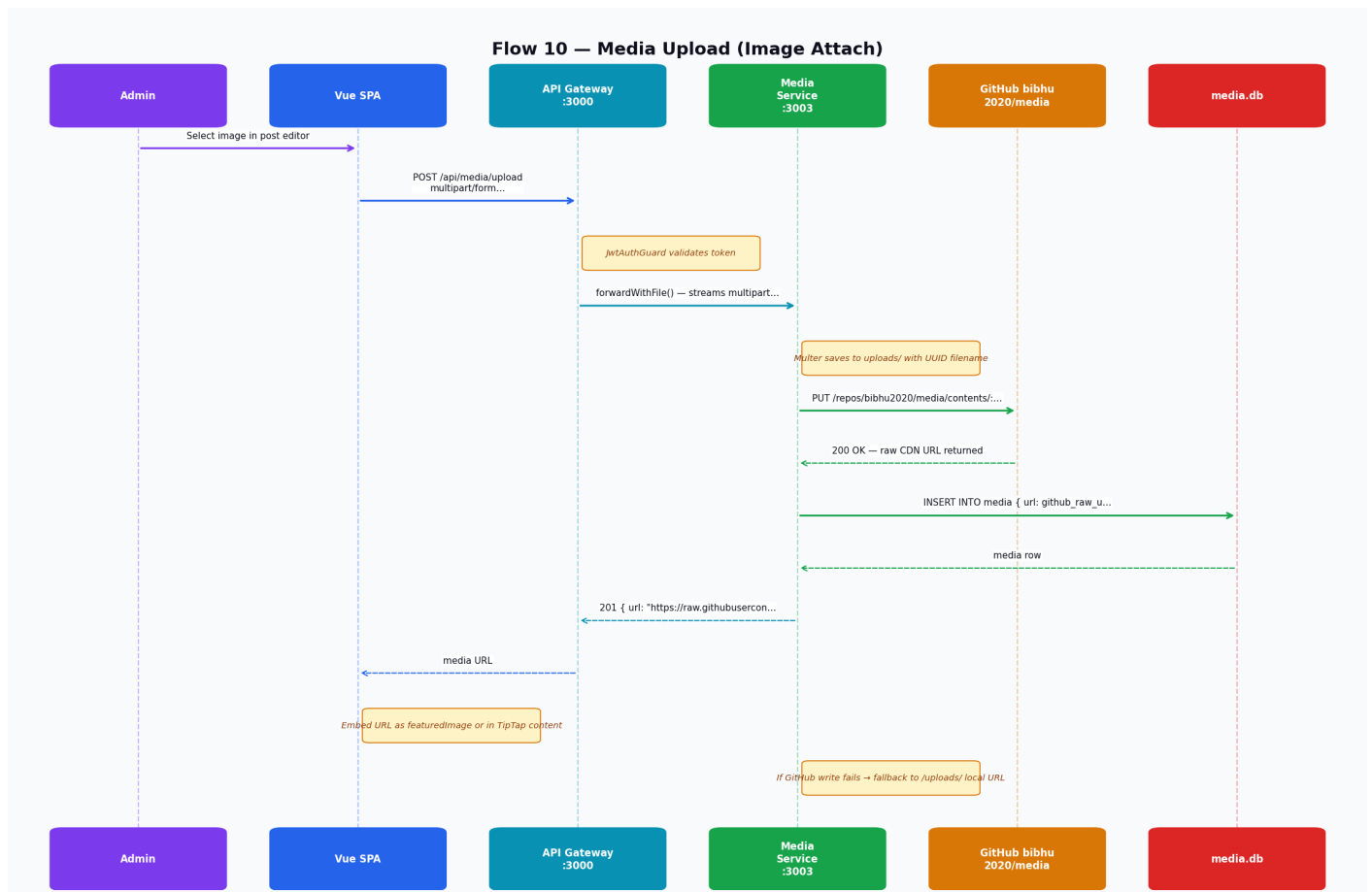


Figure 10: Media Service uploads to GitHub CDN first; if GitHub returns 403 it falls back gracefully to local /uploads/ path

6.11 AI Agent — Saturday Run (AI Trending)

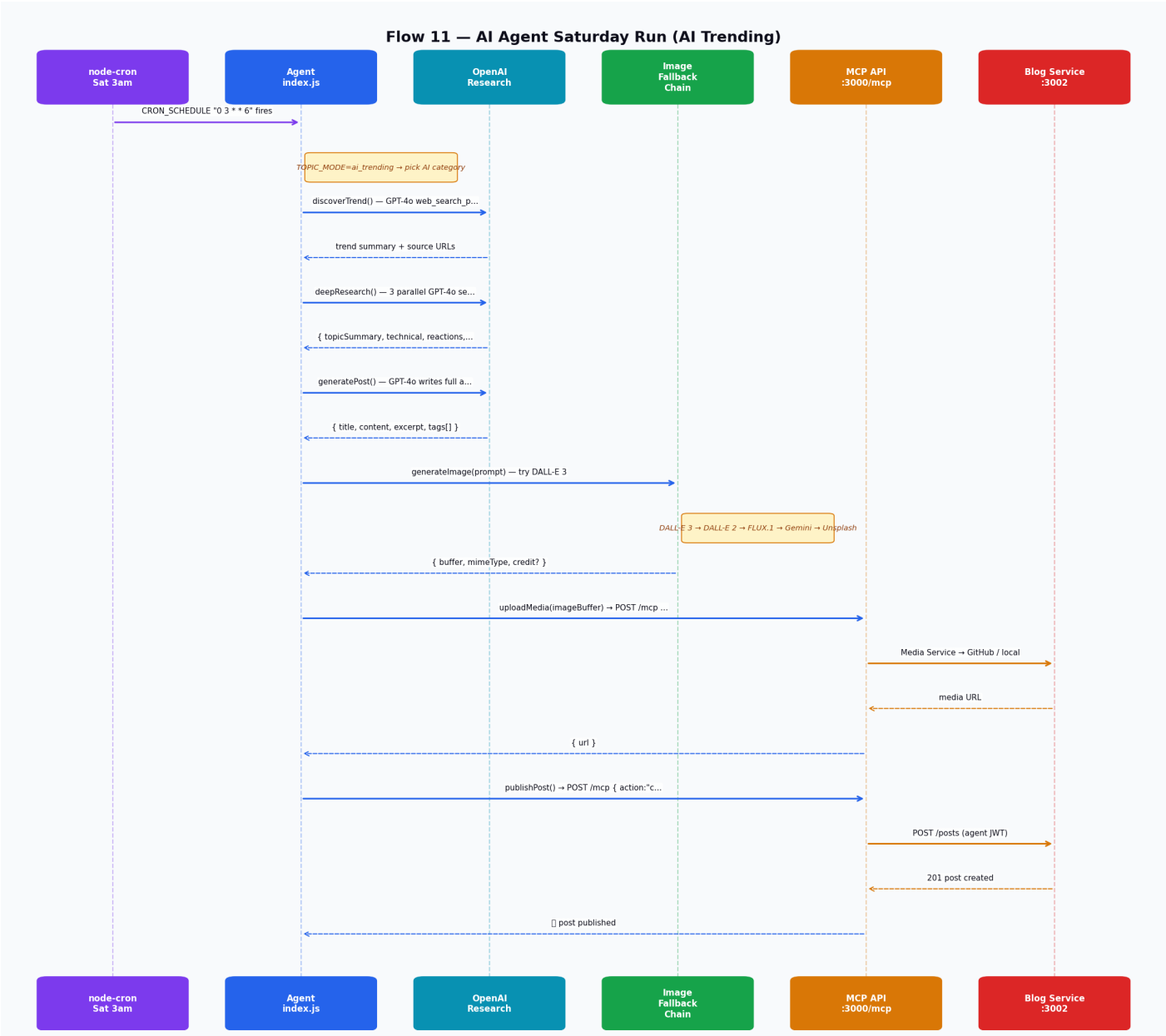


Figure 11: Saturday agent always picks the AI category. 3 parallel research queries deepen the content before GPT-4o writes the final article.

6.12 AI Agent — Sunday Run (Random Topic)

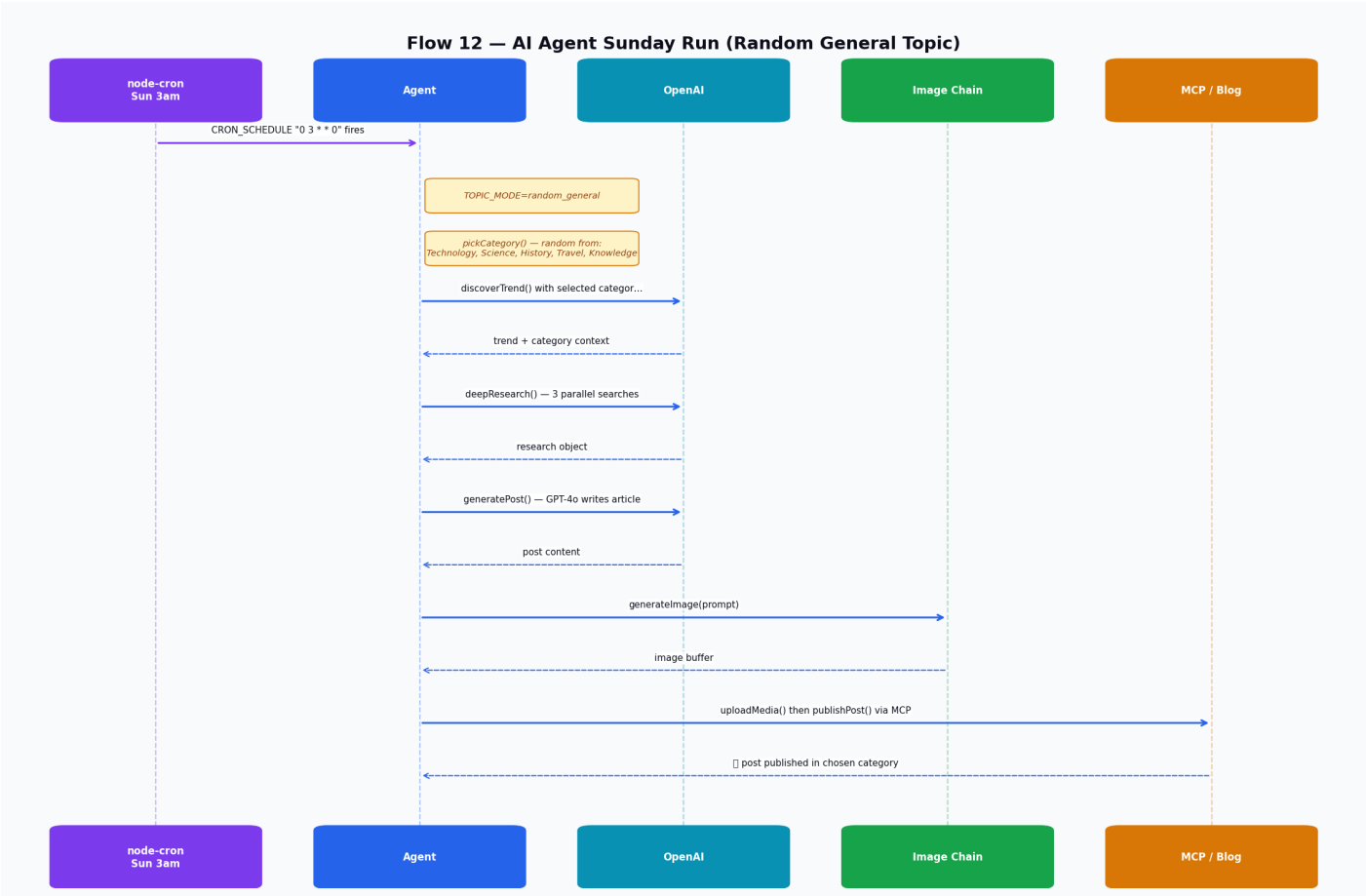


Figure 12: Sunday run picks a random non-AI category each week — ensuring content diversity across Technology, Science, History, Travel, and Knowledge

6.13 Image Generation Fallback Chain

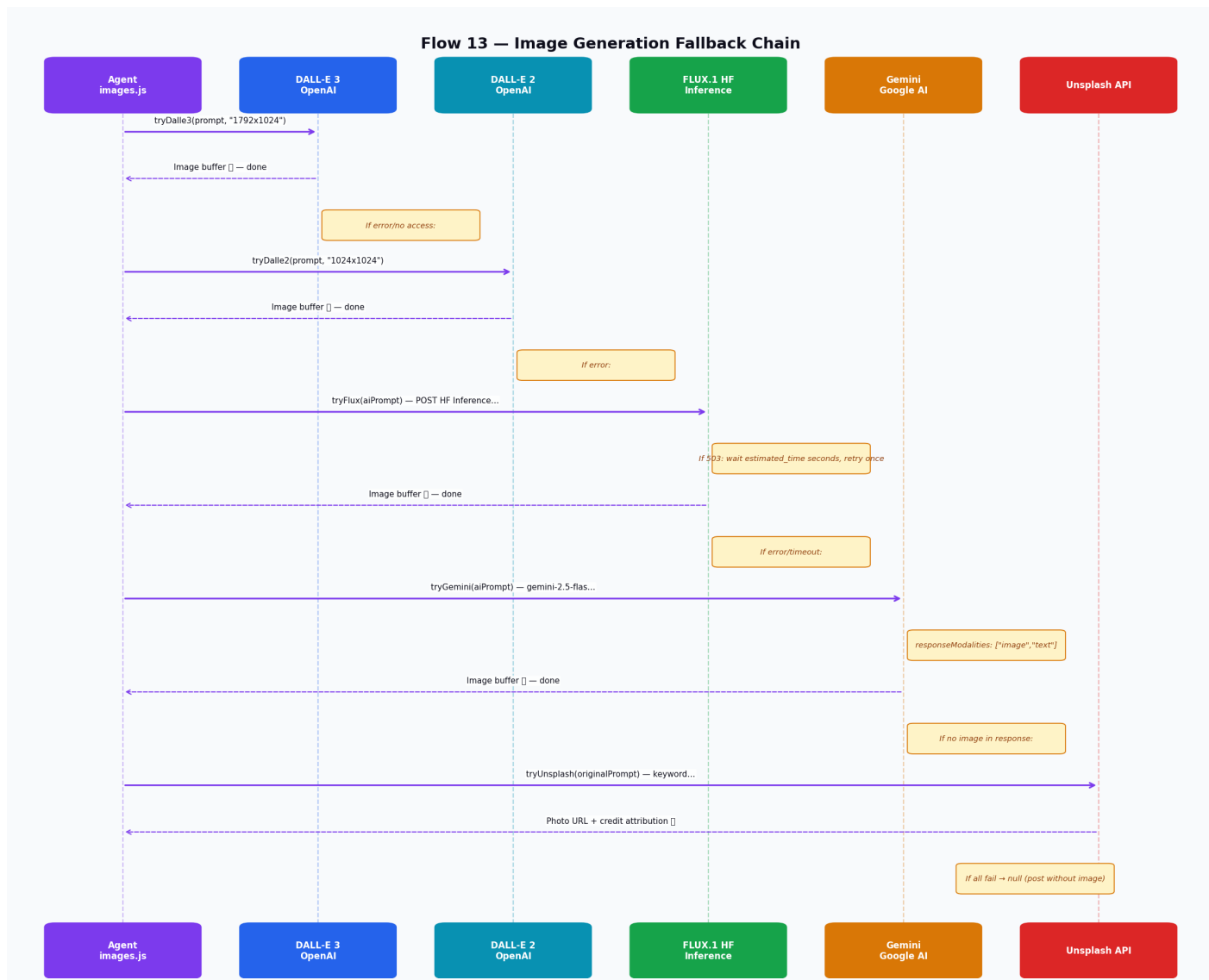


Figure 13: Five-provider fallback ensures a high-quality image is almost always found. Unsplash uses the original prompt for natural keyword extraction.

6.14 CI/CD — GitHub to HF Space Deploy

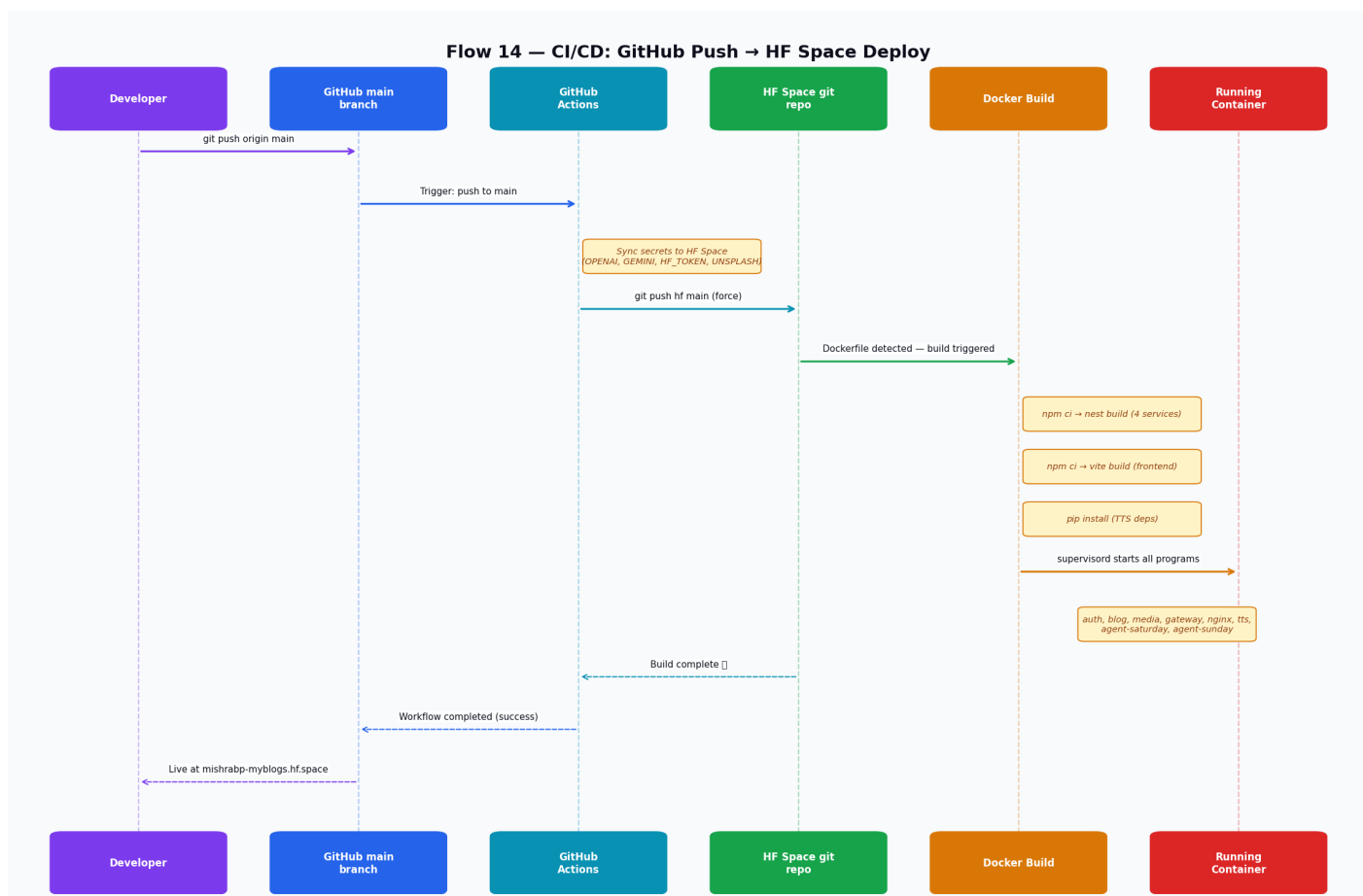


Figure 14: The deploy workflow runs in ~20s (git push only). HF Space then rebuilds the Docker image autonomously — total cold-start ~5min.

7. Technology Stack

Layer	Technology	Version / Notes
Backend Framework	NestJS	v10 — 4 independent microservices
Language	TypeScript	Strict mode; compiled to JS via tsc
Database	SQLite (better-sqlite3)	Per-service DB files; TypeORM synchronize:true
Auth	Passport JWT + bcrypt	HS256 tokens; 24h expiry; bcrypt rounds=10
Frontend	Vue 3 + Vite	Composition API; <script setup>; Vue Router; Pinia
CSS	Tailwind CSS v4	CSS-first config in style.css; @theme {} tokens
Rich Text	TipTap	StarterKit + CodeBlockLowlight + Image + Link
Syntax Highlight	highlight.js	Applied post-render on .post-content pre code
File Upload	Multer	UUID filenames; stored in media-service/uploads/
HTTP Proxy	Axios (server-side)	ProxyService.forward(); forwardWithFile() for uploads
TTS AI	Kokoro (local model)	Python subprocess; WAV output; runs on CPU
Image AI	DALL-E 3/2 → FLUX.1 → Gemini 5-providers	5-providers fallback chain

Research AI	GPT-4o + web_search_preview	OpenAI Responses API; 3 parallel queries
Article AI	GPT-4o	Writes full HTML article from research context
Scheduling	node-cron	CRON_SCHEDULE env var; two separate agent processes
Containerisation	Docker + Supervisord	Single container; multi-process managed by supervisord
Reverse Proxy	Nginx	Inside container; routes /api and /uploads
CI/CD	GitHub Actions	Push to main → sync secrets → git push to HF Space
Hosting	HuggingFace Spaces	Docker runtime; public URL: mishrabp-myblogs.hf.space
A/B Testing	localStorage + Pinia	Random variant on first visit; session-sticky